Q1 (a) Explain the elements of a Class Diagram with examples. [7 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

A **Class Diagram** is a **UML (Unified Modeling Language)** diagram that shows the **static structure** of a system. It represents **classes, their attributes, methods, and relationships** among classes.

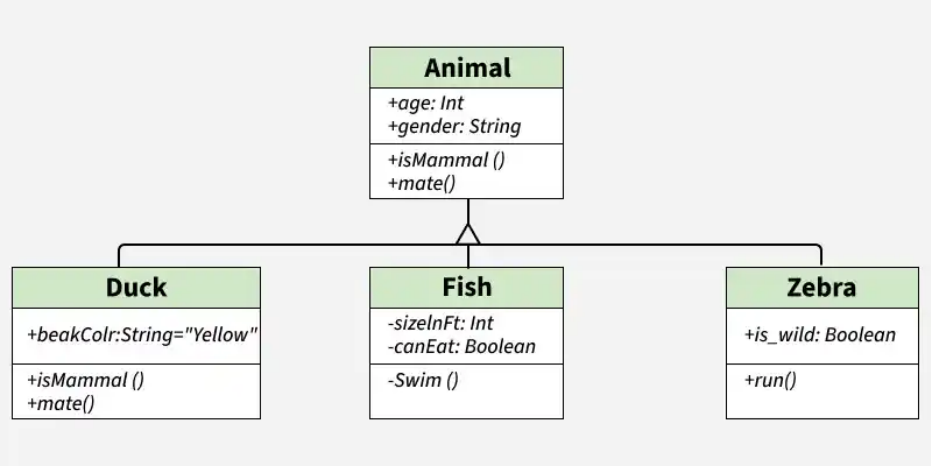
Elements of a Class Diagram

1. Class

A **class** is a blueprint or template for creating objects.

Notation:

```
+------------------+
|    Student     |
+------------------+
| rollNo : int    |
| name : String   |
+------------------+
| display()       |
| calculateMarks() |
+------------------+
```



Example:

- Student
- Customer
- ATM
- BankAccount

---

2. Attributes (Data Members)

Attributes store the **properties (data)** of a class.

Example:

For Student class:

- rollNo
- name
- age

Student

---

rollNo : int

name : String
age : int

---

3. Operations (Methods)

Operations are the **functions performed by the class**.

Example:

Student

---

display()

login()

logout()

For ATM:

- withdraw()
- deposit()
- checkBalance()

---

4. Association

Association shows the **relationship between two classes**.

Example:

Customer -------- Account

Meaning:

A customer has one or more accounts.

---

5. Multiplicity

Multiplicity tells **how many objects are associated**.

Examples:

Customer 1 -------- 1 Account

One customer has one account.

Teacher 1 -------- * Student

One teacher teaches many students.

Common multiplicities:

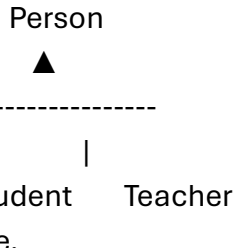
- **1** → Exactly one
- **0..1** → Zero or one
- **1..*** → One or many
- ***** → Many

---

6. Generalization (Inheritance)

Inheritance means a **child class acquires the properties of a parent class**.

Example:



Here,

- Person is the superclass.
- Student and Teacher are subclasses.

---

7. Aggregation

Aggregation represents a **"Has-A" relationship** where the child can exist independently of the parent.

Example:

Department ◇------ Professor

Even if the department is removed, the professor can still exist.

---

8. Composition

Composition is a **strong "Has-A" relationship** where the child cannot exist without the parent.

Example:

House ◆------ Room

If the house is destroyed, its rooms no longer exist independently.

---

9. Dependency

Dependency means **one class temporarily uses another class**.

Example:

Customer -------> Printer

The Customer class uses the Printer class only when printing a receipt.

Example (ATM System)

Customer ---------- ATM

| |

| |

-------- Account -----

- **Classes:** Customer, ATM, Account
- **Attributes:** accountNo, balance, PIN
- **Methods:** withdraw(), deposit(), checkBalance()
- **Association:** Customer uses ATM and owns an Account.

Advantages of Class Diagram

- Shows the static structure of the system.
- Helps understand relationships between classes.
- Makes software design easy.
- Improves maintenance and code reuse.
- Acts as a blueprint before coding.

University Exam Definition (Write First)

A Class Diagram is a UML diagram that represents the static structure of a system by showing classes, their attributes, operations (methods), and relationships such as association, inheritance, aggregation, composition, and dependency.

Q1 (b) What do you mean by Association Class? [3–5 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

An **Association Class** is a special class in UML that is attached to an **association (relationship)** between two classes. It is used when the **relationship itself has its own attributes or operations**.

In simple words:

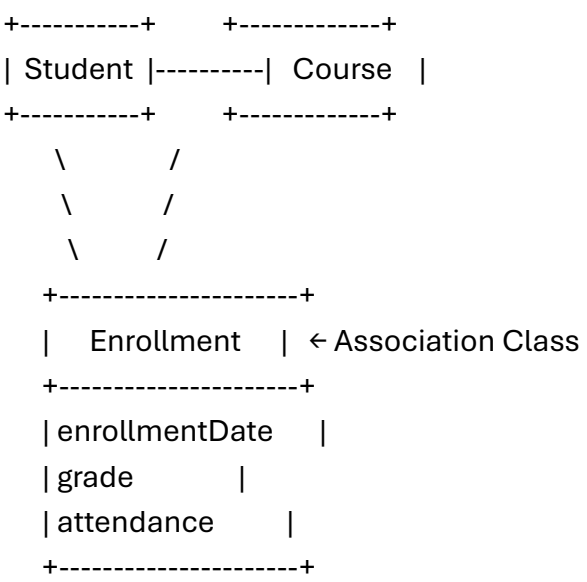
When the relationship between two classes contains additional information, we use an Association Class.

Example

Consider **Student** and **Course**.

- A student enrolls in a course.
- The **enrollment** has its own information like:
    - Enrollment Date
    - Grade
    - Attendance

These details belong to the relationship, not only to Student or Course.



Why is an Association Class Used?

- To store information about the relationship.
- To add attributes and methods to an association.
- To make the UML model more clear and organized.

Real-Life Example

- **Doctor** treats **Patient**.
- The treatment relationship has:
    - Treatment Date
    - Prescription
    - Fees

Here, **Treatment** is an **Association Class**.

Advantages

- Represents relationship data clearly.
- Avoids duplication of information.
- Improves the design of the system.

Exam Definition (Write First)
An Association Class is a UML class that is attached to an association between two classes to represent attributes and operations of the relationship itself.

Q1 (c) Explain Different Relations in UML. [7 Marks]
(SPPU BE OOMD – Simple Answer)
Definition
Relationships (Relations) in UML show how different classes or objects are connected and interact with each other in a system.
There are **five main types of relationships in UML**:

1. Association
2. Aggregation
3. Composition
4. Generalization (Inheritance)
5. Dependency

UML relations are logical connections between different classes and objects, including the primary types of Dependency, Association, and Generalization. These connections define how elements interact, share behavior, and depend on each other within a system model. [1, 2, 3]
Core UML Relationships

- **Dependency**
  - A temporary or weak "using" relationship where a change in one class (supplier) may cause changes in the other class (client).
  - Represented by a dashed line with an open arrow pointing to the independent class. [1, 2]
- **Association**
  - A structural link connecting two peer classes that communicate with each other, representing a "has-a" or general connection.
- Represented by a solid line between the classes, which can include multiplicity (e.g., 1 to 0..*). [1, 2, 3, 4, 5]
- **Aggregation**
  A specialized, whole-part association where the "part" class can exist independently if the "whole" container class is destroyed.
  Represented by a solid line with an unfilled diamond shape next to the whole or parent class. [1, 2]
- **Composition**
  A stronger form of aggregation signifying strict ownership, where the "part" cannot survive without the "whole" container.

Represented by a solid line with a filled/solid diamond shape next to the parent class. [1, 2]

- **Generalization (Inheritance)**
  An "is-a" connection where a specialized child class inherits the attributes and methods of a generalized parent class.
  Represented by a solid line with a hollow/unfilled arrowhead pointing from the child to the parent. [1, 2, 3]

Realization (Implementation)
A relationship where one class specifies a contract or public interface, and another class implements the operations defined by that interface.
Represented by a dashed line with a hollow closed arrowhead pointing to the interface. [1, 2, 3, 4]

Q2 (a) What are different types of Analysis Classes? Explain with an example. [6 Marks]
Definition
During the **Object-Oriented Analysis (OOA)** phase, the system requirements are analyzed to identify the important classes. These classes are known as **Analysis Classes**. They represent the structure and behavior of the system before the design and coding phases.
Analysis classes help developers understand the responsibilities of different parts of the system and separate the user interface, business logic, and data storage.
There are **three types of Analysis Classes**:

1. Entity Class
2. Boundary Class
3. Control Class

---

1. Entity Class
Definition
An **Entity Class** represents the **data or information** of the system. It stores the important business data that needs to be maintained throughout the system. These classes usually correspond to database tables and have a longer life than other classes.
Characteristics

- Stores business information.
- Represents real-world objects.
- Exists for a long time.
- Can be stored in a database.
- Mostly contains attributes and business-related methods.

Examples

- Student
- Employee
- Customer
- BankAccount
- Product

ATM Example
Entity Class

BankAccount

AccountNo
Balance
PIN
AccountType

---

deposit()
withdraw()
checkBalance()

---

2. Boundary Class

Definition

A **Boundary Class** represents the **interaction between the user and the system**. It provides the interface through which users enter data and receive output. It acts as a communication link between the user and the control class.

Characteristics

- Accepts user input.
- Displays output to the user.
- Represents forms, screens, web pages, or APIs.
- Does not contain business logic.
- Communicates with the Control Class.

Examples

- Login Screen
- ATM Display Screen
- Registration Form
- Payment Page

ATM Example

Boundary Class

ATM Screen

EnterPIN()
DisplayMenu()
ShowBalance()
PrintReceipt()

---

3. Control Class

Definition

A **Control Class** controls the execution of the system. It contains the **business logic** and coordinates communication between Boundary and Entity classes. It decides what actions should be performed when a user requests a service.

Characteristics

- Controls the flow of the application.

- Implements business rules.
- Coordinates between Boundary and Entity classes.
- Usually created for each use case.

Examples

- LoginController
- PaymentController
- TransactionController
- RegistrationController

ATM Example

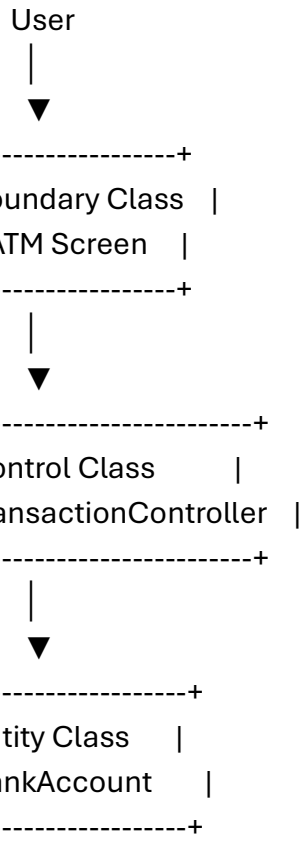
Control Class

TransactionController

---

verifyPIN()
withdrawMoney()
depositMoney()
transferFunds()

Working of Analysis Classes (ATM Example)



Explanation

1. The user inserts the ATM card and enters the PIN through the **ATM Screen (Boundary Class)**.
2. The request is sent to the **TransactionController (Control Class)**.
3. The controller verifies the PIN by accessing the **BankAccount (Entity Class)**.
4. If the PIN is correct, the requested transaction is performed.
5. Finally, the result is displayed on the ATM Screen.

Difference Between Analysis Classes

Entity Class	Boundary Class	Control Class
Stores business data	Interacts with the user	Controls application flow
Represents real-world objects	Represents screens/forms	Represents business logic
Long life	Temporary	Temporary
Connected to database	Connected to user	Connected to both Boundary and Entity
Example: BankAccount	Example: ATM Screen	Example: TransactionController

---

Advantages of Analysis Classes

- Separates data, interface, and business logic.
- Makes the system modular and easy to understand.
- Improves software maintenance.
- Simplifies testing and debugging.
- Encourages code reusability.
- Helps in converting analysis models into design models

Q2 (b) How do you model Class & Object in UML? [5 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

In **UML (Unified Modeling Language)**, a system is modeled using **Classes** and **Objects**.

- A **Class** represents a blueprint or template for creating objects.
- An **Object** is an instance of a class that contains actual values.

Modeling classes and objects helps developers understand the structure of the system before coding.

---

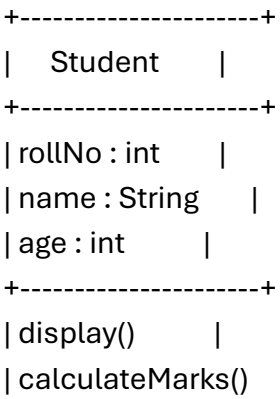
1. Modeling a Class

A **Class** is represented by a **rectangle divided into three compartments**.

Compartments

1. Class Name
2. Attributes (Data Members)
3. Operations (Methods)

UML Notation



```
+----------------------+
```

Explanation

- **Class Name:** Student
- **Attributes:** rollNo, name, age
- **Methods:** display(), calculateMarks()

---

2. Modeling an Object

An **Object** is represented by a rectangle with the **object name underlined**. It contains actual values of the attributes.

UML Notation

```
+----------------------+
| __student1:Student__ |
+----------------------+
| rollNo = 101         |
| name = Rahul         |
| age = 20             |
+----------------------+
```

Explanation

- **student1** → Object name
- **Student** → Class name
- **101, Rahul, 20** → Actual values stored in the object

Difference Between Class and Object

Class	Object
Blueprint or template	Instance of a class
Defines attributes and methods	Contains actual data
No memory allocated until object is created	Memory is allocated
Example: Student	Example: student1

Example (ATM System)

Class

```
+----------------------+
|   BankAccount        |
+----------------------+
| accountNo            |
| balance              |
| PIN                  |
+----------------------+
| deposit()            |
| withdraw()           |
| checkBalance()       |
+----------------------+
```

Object

```
+---------------------------+
```

| __acc1 : BankAccount__ |
+---------------------------+
| accountNo = 12345 |
| balance = ₹50,000 |
| PIN = 1234 |
+---------------------------+

Advantages

- Provides a clear structure of the system.
- Helps identify attributes and methods.
- Improves communication among developers.
- Simplifies software design and maintenance.

Exam Definition (Write First)

In UML, a Class is modeled as a rectangle with three compartments (name, attributes, and operations), while an Object is modeled as an instance of a class containing actual attribute values.

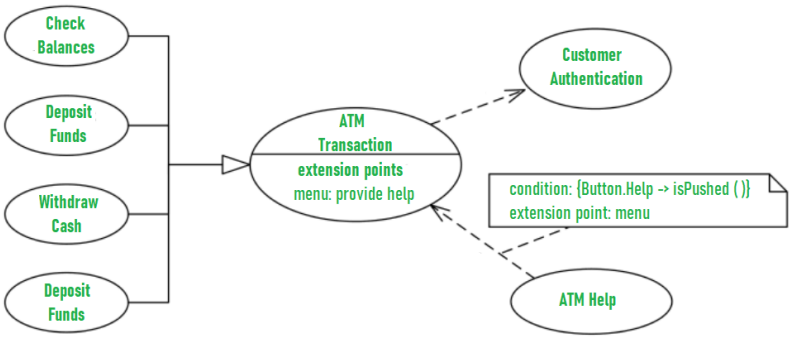
Q2 (c) Draw a Class Diagram for ATM System. [6 Marks]

(SPPU BE OOMD – Simple Answer)

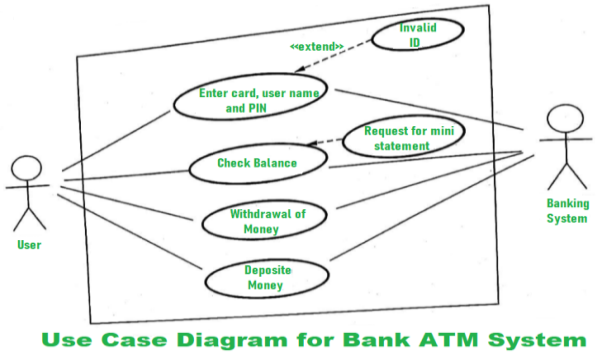
Definition

A **Class Diagram** is a UML diagram that represents the **static structure** of the ATM system. It shows the classes, their attributes, methods, and relationships.

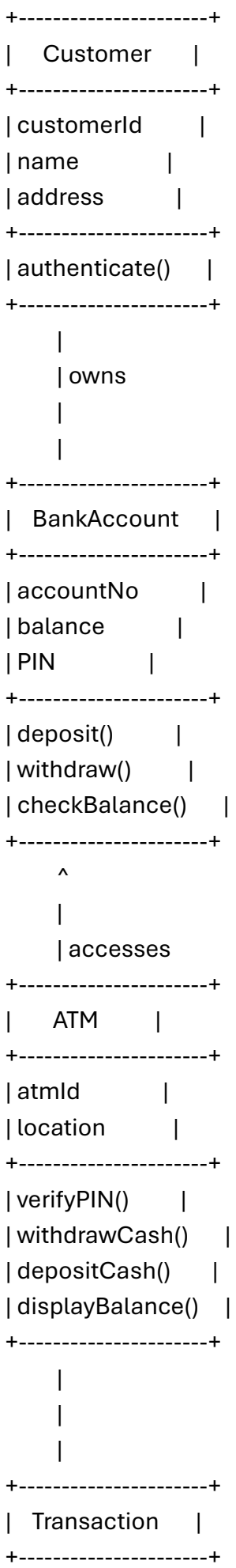
ATM Class Diagram



Use Case Diagram for Customer Authentication



Use Case Diagram for Bank ATM System


Customer
customerId
name
address
authenticate()
owns
BankAccount
accountNo
balance
PIN
deposit()
withdraw()
checkBalance()
accesses
ATM
atmId
location
verifyPIN()
withdrawCash()
depositCash()
displayBalance()
Transaction

```
| transactionId      |
| date               |
| amount             |
+--------------------+
| execute()          |
| printReceipt()     |
+--------------------+
```

Explanation of Classes

1. Customer

- Represents the ATM user.
- Owns one or more bank accounts.

2. BankAccount

- Stores account details such as account number, balance, and PIN.
- Performs deposit, withdrawal, and balance inquiry operations.

3. ATM

- Accepts user requests.
- Verifies the PIN.
- Communicates with the BankAccount class.

4. Transaction

- Maintains transaction details.
- Executes the requested operation and generates the receipt.

Relationships

- **Customer → BankAccount:** Association (Customer owns an account)
- **ATM → BankAccount:** Association (ATM accesses account information)
- **ATM → Transaction:** Association (ATM performs transactions)

Advantages

- Shows the complete structure of the ATM system.
- Identifies classes and their responsibilities.
- Helps developers during software design.
- Improves maintainability and reusability.

Q1 (a) Explain the types of Relationship along with their notations. [6 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

In UML, a **relationship** represents the connection or interaction between two or more classes or objects. Relationships help to show how classes communicate, depend on, or inherit from each other.

The main types of UML relationships are:

1. Association
2. Aggregation
3. Composition
4. Generalization (Inheritance)

5. Dependency

---

1) Association Relationship

Definition

Association is a **structural relationship** that shows a connection between two classes. It represents that objects of one class are connected with objects of another class.

Notation

Class A ---------- Class B

(Simple line)

Example:

Customer ---------- Account

A customer has an account.

Types of Association:

- One-to-One (1:1)
- One-to-Many (1:*)
- Many-to-Many (:)

Example:

Teacher 1 ---------- * Student

One teacher teaches many students.

---

2) Aggregation Relationship

Definition

Aggregation is a **special type of association** that represents a weak "Has-A" relationship. The child object can exist independently of the parent object.

Notation

Class A ◇---------- Class B

(Hollow diamond)

Example:

Department ◇---------- Professor

A department has professors, but professors can exist even if the department is removed.

Characteristics:

- Represents whole-part relationship.
- Objects have independent life cycles.

---

3) Composition Relationship

Definition

Composition is a **strong form of aggregation** where the child object cannot exist without the parent object.

Notation

Class A ◆---------- Class B

(Filled diamond)

Example:

House ◆---------- Room

If the house is destroyed, rooms also cease to exist.

Characteristics:

- Strong ownership.
- Child depends completely on the parent.

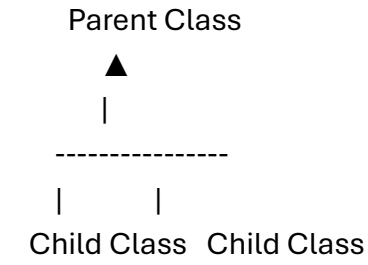
---

4) Generalization (Inheritance) Relationship

Definition

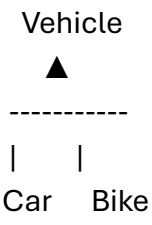
Generalization represents an **"IS-A" relationship** where a child class inherits attributes and methods from a parent class.

Notation



(Empty triangle arrow)

Example:



Car and Bike inherit properties from Vehicle.

---

5) Dependency Relationship

Definition

Dependency is a relationship where one class **uses another class temporarily**. A change in one class may affect the dependent class.

Notation

Class A - - - - - - > Class B

(Dashed arrow)

Example:

Customer - - - - - > Printer

Customer uses Printer to print a receipt.

Summary Table

Relationship	Meaning	UML Notation	Example
Association	Connection between classes	———	Customer–Account
Aggregation	Weak Has-A relationship	◇———	Department–Professor
Composition	Strong Has-A relationship	◆———	House–Room
Generalization	Is-A relationship	△———	Car–Vehicle
Dependency	Temporary usage	- - - →	Customer–Printer

Conclusion

UML relationships are used to represent the interaction and dependency between classes. Association, Aggregation, Composition, Generalization, and Dependency are the most commonly used relationships in object-oriented modeling.

Q1 (b) Draw Class Diagram for ATM System. [6 Marks]

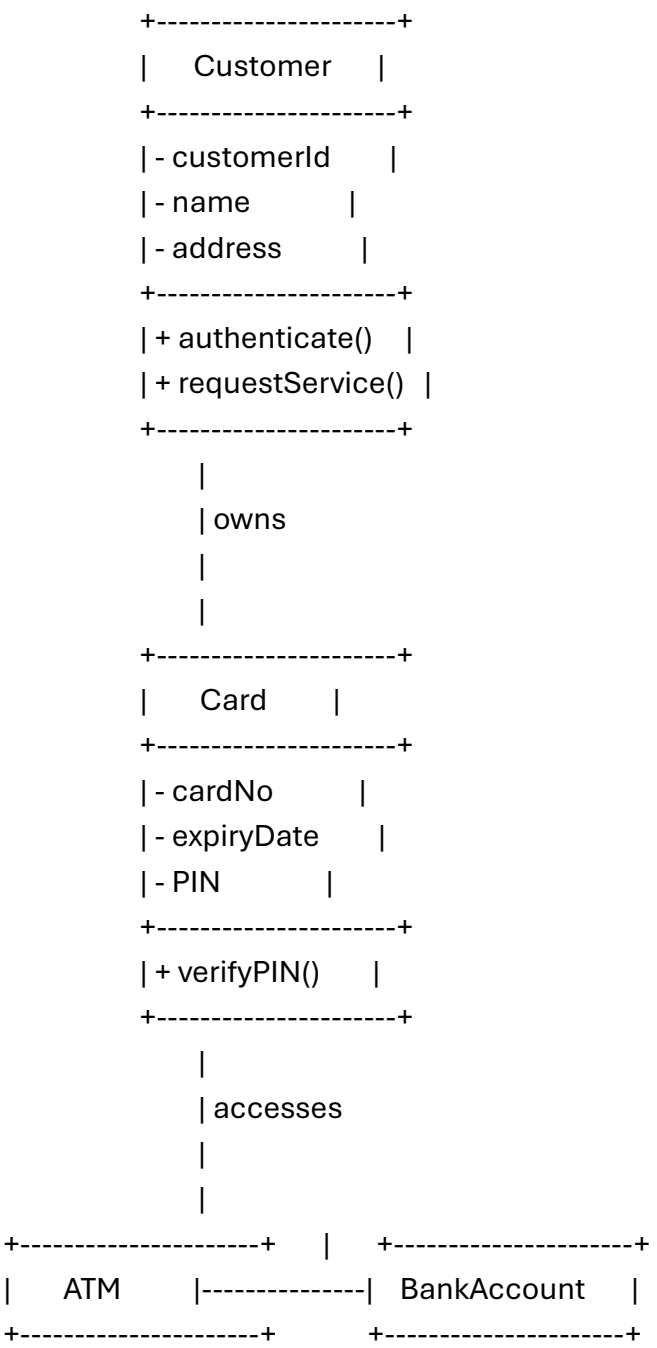
(SPPU BE OOMD – Simple Answer)

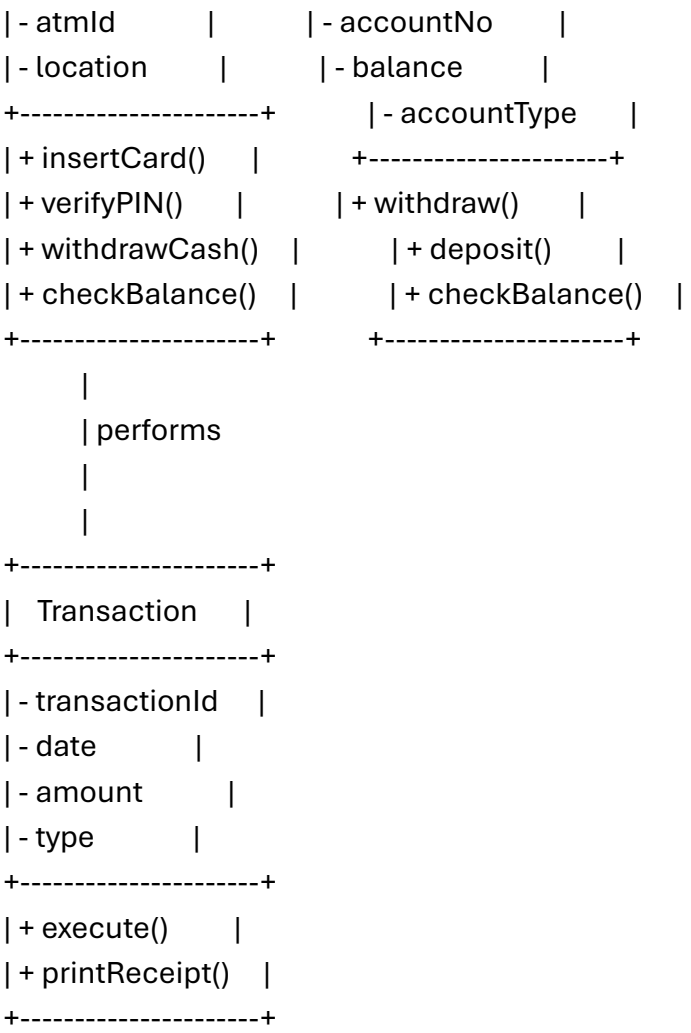
Definition

A **Class Diagram** is a UML diagram that represents the **static structure of a system**. It shows classes, their attributes, methods, and relationships among classes.

An ATM system consists of classes such as **Customer, ATM, Bank Account, Card, and Transaction**.

UML Class Diagram for ATM System





Explanation of Classes

1. Customer Class

- Represents the ATM user.
- Contains customer details.
- Performs authentication.

Attributes:

- customerId
- name
- address

Methods:

- authenticate()
- requestService()

---

2. ATM Class

- Represents the ATM machine.
- Provides services to customers.

Attributes:

- atmId
- location

Methods:

- insertCard()

- verifyPIN()
- withdrawCash()
- checkBalance()

---

3. Card Class

- Stores card information.
- Used for customer authentication.

Attributes:

- cardNo
- expiryDate
- PIN

Method:

- verifyPIN()

---

4. BankAccount Class

- Stores account details and balance information.

Attributes:

- accountNo
- balance
- accountType

Methods:

- withdraw()
- deposit()
- checkBalance()

---

5. Transaction Class

- Maintains transaction details.

Attributes:

- transactionId
- date
- amount
- type

Methods:

- execute()
- printReceipt()

Relationships in ATM Class Diagram

1. **Customer – Card**
    - Association relationship
    - Customer uses a card for ATM transactions.
2. **Card – BankAccount**
    - Association relationship
    - Card is linked with a bank account.
3. **ATM – BankAccount**
    - Association relationship

- ATM accesses account information.

4. **ATM – Transaction**
   - Association relationship
   - ATM performs transactions.

Conclusion

The ATM class diagram represents the static view of the ATM system by identifying classes, attributes, operations, and relationships. It helps in designing and implementing the ATM software system.

Q1 (c) Define Object? Explain components of Object Diagram in brief. [6 Marks]

(SPPU BE OOMD – Simple Answer)

Definition of Object

An Object is an instance of a class that represents a real-world entity. An object contains state (attributes/data) and behavior (methods/functions).

In UML, an object represents a specific instance of a class with actual values assigned to its attributes.

Example:

A class Student can have objects:

- **Student1**
- **Student2**

Where:

Student1

Name = Rahul

RollNo = 101

Age = 20

Here, Student1 is an object of the Student class.

Object Diagram

Definition

An Object Diagram is a UML structural diagram that represents the snapshot of objects in a system at a particular point of time.

It shows:

- **Objects**
- **Their attributes with values**
- **Relationships between objects**

Object diagrams are used to understand how objects interact during execution.

Components of Object Diagram

1. Object

Definition:

An object represents an instance of a class.

Notation:

+----------------------+

| student1 : Student |
+----------------------+

Where:

- **student1 → Object name**
- **Student → Class name**

---

2. Object Name
Definition:
Object name identifies a particular object in the system.
Notation:
objectName : ClassName
Example:
customer1 : Customer
Here, customer1 is the object name.

---

3. Attributes and Values
Definition:
Attributes represent the properties of an object with their current values.
Example:

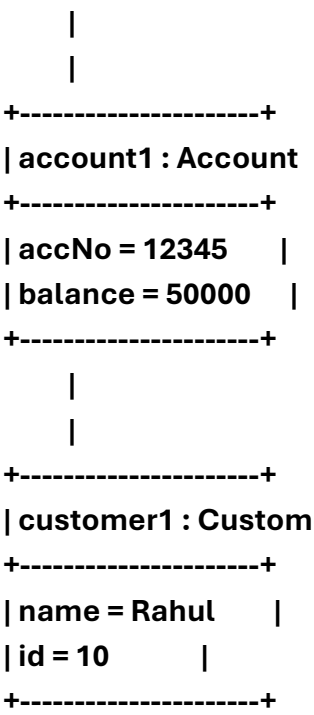
+----------------------+
| customer1 : Customer |
+----------------------+
| name = Amit |
| id = 101 |
| city = Pune |
+----------------------+

---

4. Links (Object Relationships)
Definition:
A link represents a relationship between two objects. It is an instance of an association between classes.
Notation:
Object A ---------- Object B
Example:
Customer1 ---------- Account1
It shows that customer1 is connected with account1.

Example of Object Diagram (ATM System)

+----------------------+
| atm1 : ATM |
+----------------------+
| atmId = 101 |
| location = Pune |
+----------------------+
|



Difference Between Class Diagram and Object Diagram

Class Diagram	Object Diagram
Shows classes of a system	Shows objects of a system
Represents design view	Represents runtime view
Contains attributes and methods	Contains actual attribute values
Blueprint of the system	Snapshot of the system

Advantages of Object Diagram

- **Shows the real-time view of the system.**
- **Helps understand object relationships.**
- **Useful for testing and debugging.**
- **Helps verify class diagram design.**

Exam Definition (Write First)

An object is an instance of a class that has its own identity, state, and behavior. An object diagram represents a set of objects and their relationships at a particular point of time.

Q2 (a) Differentiate between Link and Association with suitable example. [6 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

Association

An Association is a relationship between two or more classes. It describes how classes are connected in a system.

Link

A Link is a connection between objects. It is an instance of an association and represents the relationship between specific objects at runtime.

Difference between Link and Association

Association	Link
Association is a relationship between classes.	Link is a relationship between objects.
It represents the static structure of the system.	It represents the runtime (instance) connection between objects.
It is defined during system design.	It exists when objects are created during execution.
One association can have many links.	A link belongs to only one association.
Represented in a Class Diagram.	Represented in an Object Diagram.
Example: Customer – Account	Example: customer1 – account1

Example of Association

A Customer class is associated with a BankAccount class.

```
+-----------+          +--------------+
| Customer  |----------| BankAccount  |
+-----------+          +--------------+
```

Explanation:

This shows that a Customer can have a BankAccount. It is a relationship between classes.

Example of Link

Objects of the above classes are connected as follows:

```
+---------------------+     +--------------------------+
| customer1:Customer  |------| account1:BankAccount     |
+---------------------+     +--------------------------+
```

Explanation:

Here, customer1 is connected to account1. This is a Link because it connects specific objects.

Relationship between Association and Link

Class Diagram

```
Customer ---------------- BankAccount
    |
    | Association
    ▼
```

Object Diagram

```
customer1 -------- account1
    |
    | Link
    ▼
```

Explanation:

- **Association defines the relationship between classes.**
- **Link is the actual connection between their objects.**

Conclusion

Association represents a general relationship between classes, whereas a Link represents the actual relationship between specific objects created from those classes.

Q. What is Object-Oriented Development (OOD)? Explain the OO Themes. [6 Marks]

(SPPU BE OOMD – Simple Answer)

Object-Oriented Development (OOD)

Definition

Object-Oriented Development (OOD) is a software development approach in which a system is developed using objects and classes. It follows the principles of object-oriented programming such as abstraction, encapsulation, inheritance, and polymorphism to design and implement software.

OOD divides the software into small, reusable objects that interact with each other to perform specific tasks.

Advantages of OOD

- **Improves code reusability.**
- **Makes software easy to maintain.**
- **Reduces development time.**
- **Provides better security through encapsulation.**
- **Makes software scalable and flexible.**

OO Themes (Characteristics of Object Orientation)

OO themes are the basic principles on which Object-Oriented Development is based.

1. Abstraction

Definition

Abstraction means showing only the essential features of an object while hiding unnecessary implementation details.

Example

In an ATM, the user only sees options like Withdraw, Deposit, and Balance Inquiry. The internal processing is hidden.

---

2. Encapsulation

Definition

Encapsulation is the process of combining data (attributes) and methods (functions) into a single unit (class) and protecting the data from unauthorized access.

Example

A BankAccount class stores the account balance and allows access only through methods such as deposit() and withdraw().

---

3. Modularity

Definition

Modularity means dividing a software system into independent modules or classes, where each module performs a specific task.

Example

In an ATM system:

- **Customer Class**
- **ATM Class**
- **BankAccount Class**
- **Transaction Class**

Each class performs a separate function.

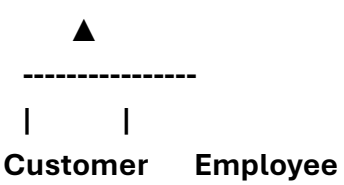
---

4. Hierarchy

Definition

Hierarchy organizes classes into parent-child relationships using inheritance.

Example



Customer and Employee inherit common properties from Person.

---

5. Typing

Definition

Typing means every object belongs to a particular class, and only valid operations can be performed on that object.

Example

A Student object can use only the methods defined in the Student class.

---

6. Concurrency

Definition

Concurrency means multiple objects can execute different tasks simultaneously.

Example

In an ATM system:

- **One customer withdraws money.**
- **Another customer checks the balance.**

Both operations can occur at the same time.

---

7. Persistence

Definition

Persistence means objects continue to exist even after the program stops by storing their data permanently in a database or file.

Example

Bank account details remain stored in the database even after the ATM session ends.

Q. Explain the following terms with respect to Object-Oriented Methodology. [5 Marks]
(SPPU BE OOMD – Simple Answer)

i) Abstraction

Definition

Abstraction is the process of showing only the essential features of an object while hiding unnecessary implementation details. It allows the user to focus on what an object does rather than how it works.

Example

In an ATM, the user can perform operations like Withdraw, Deposit, and Check Balance without knowing the internal processing.

Advantages

- **Reduces complexity.**
- **Improves security.**
- **Makes software easy to understand.**

---

ii) Encapsulation

Definition

Encapsulation is the process of combining data (attributes) and methods (functions) into a single unit called a class. It also protects data from unauthorized access.

Example

A BankAccount class stores the balance as private and allows access only through methods like deposit() and withdraw().

BankAccount

- balance

+ deposit()

+ withdraw()

Advantages

- **Protects data.**
- **Improves security.**
- **Makes maintenance easier.**

---

iii) Sharing

Definition

Sharing means reusing common properties and methods among different classes or objects. It is mainly achieved through inheritance, where a child class shares the features of its parent class.

Example

Person

▲

| |

Student Teacher

Here, Student and Teacher share common properties (such as name and age) inherited from the Person class.

Advantages

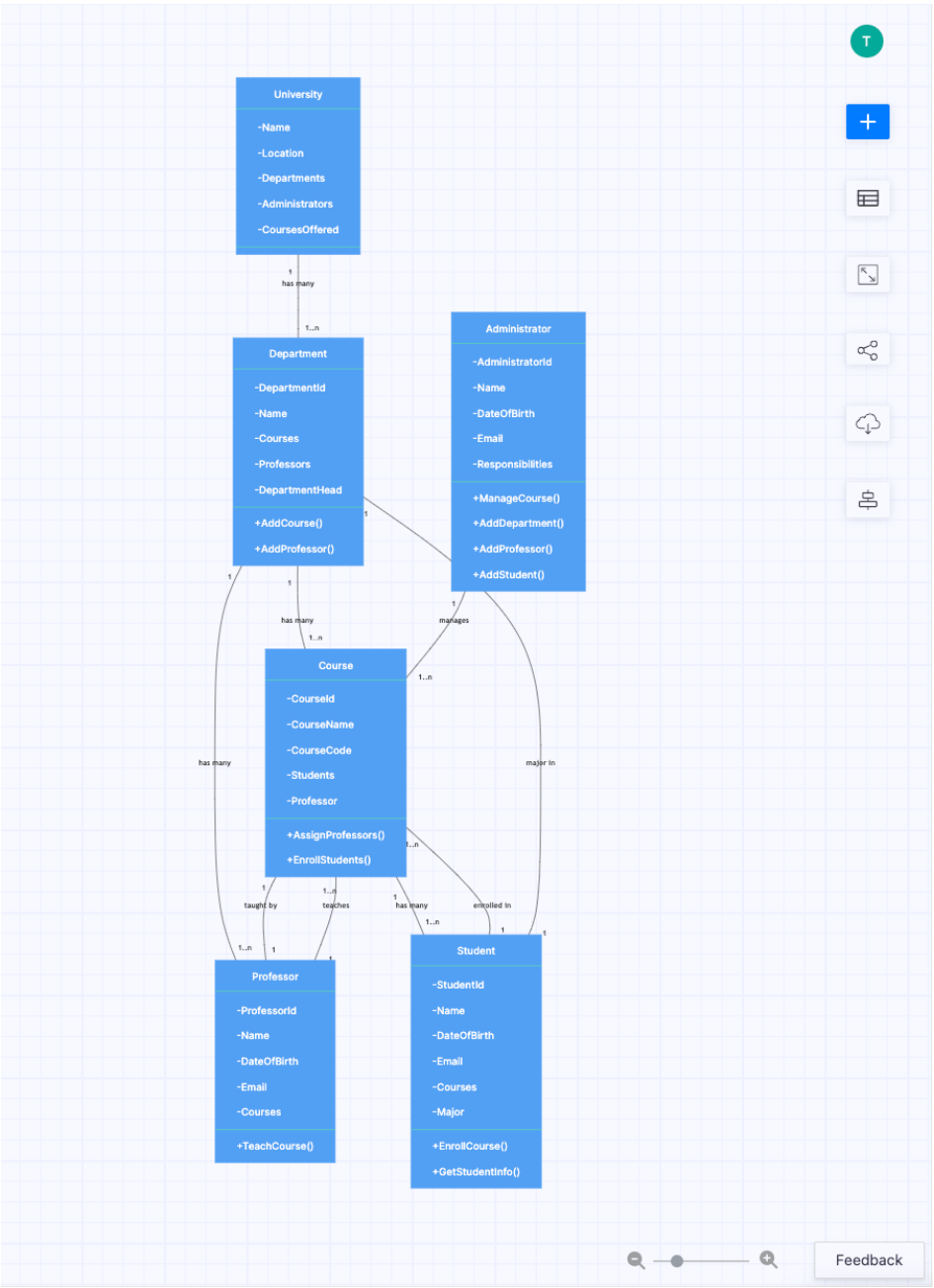
- **Promotes code reusability.**
- **Reduces code duplication.**
- **Simplifies software development and maintenance.**

Conclusion

- **Abstraction → Hides unnecessary details.**
- **Encapsulation → Combines data and methods while protecting data.**
- **Sharing → Reuses common features through inheritance.**

Q1 (a) Draw a Class Diagram for University Database System. [6 Marks]

UML Class Diagram



Faculty -------- receives --------> Contract

Faculty -------- receives --------> Acknowledgement

Explanation of Classes

1. College
Contains multiple academic departments.
2. Department
Manages students, faculty members, and graduate students.
3. Student
Stores student information and registers for courses.
4. Transcript
Maintains the academic record and grades of each student.
5. Registration
Stores details of course registration.
6. Course
Represents university courses.
7. Section
Each course may have several sections.
8. Instructor
Teaches one or more course sections.
9. Faculty
Works in a department, teaches courses, and supervises research projects.
10. GraduateStudent
Participates in sponsored research projects.
11. ResearchProject
Stores sponsored research project details.
12. Contract & Acknowledgement
Stores contracts awarded and acknowledgements received by faculty members.

Relationships

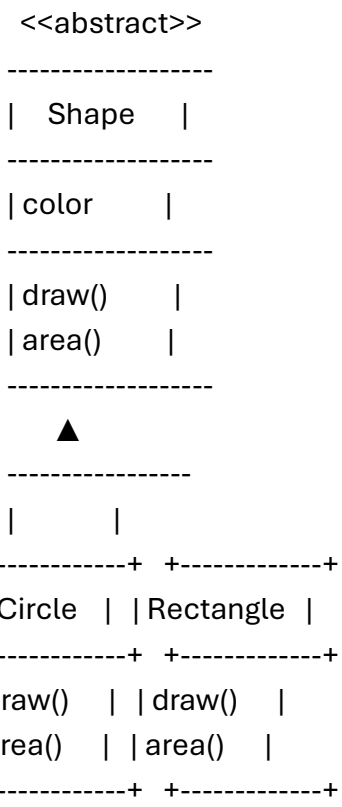
- **College has Departments.**
- **Department has Students and Faculty.**
- **Student has Transcript and Registration.**
- **Student registers for Courses.**
- **Course has many Sections.**
- **Section is taught by an Instructor.**
- **Faculty supervises Research Projects.**
- **Graduate Students work on Research Projects.**
- **Faculty receives Contracts and Acknowledgements.**

Q1 (b) What do you mean by Abstract Class? Explain with example. [4 Marks]
(SPPU BE OOMD – Simple Answer)
Definition
An **Abstract Class** is a class that **cannot be instantiated (cannot create objects)**. It is used as a **base class** and contains **abstract methods (methods without implementation)** as well as **normal methods (methods with implementation)**.
It is mainly used to provide a common structure for its subclasses.

Characteristics of Abstract Class

- An abstract class **cannot have objects**.
- It contains **abstract and concrete (normal) methods**.
- It is inherited by other classes.
- Child classes must implement all abstract methods.
- It promotes **code reusability** and **abstraction**.

UML Notation



Example

Suppose **Shape** is an abstract class.

- **Shape** contains abstract methods **draw()** and **area()**.
- **Circle** and **Rectangle** inherit from **Shape**.
- Both classes provide their own implementation of **draw()** and **area()**.

Advantages

- Provides a common base class.
- Reduces code duplication.
- Supports abstraction.
- Improves code reusability.
- Makes software easy to maintain.

Conclusion

An **Abstract Class** provides a common template for related classes. It cannot be instantiated directly but is inherited by subclasses, which implement its abstract methods.

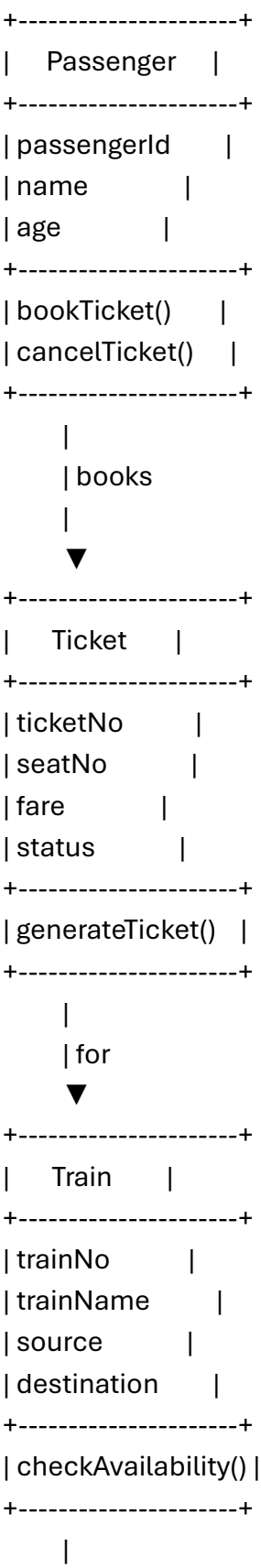
Q1 (c) Draw a Class Diagram for Railway Reservation System. [5 Marks]

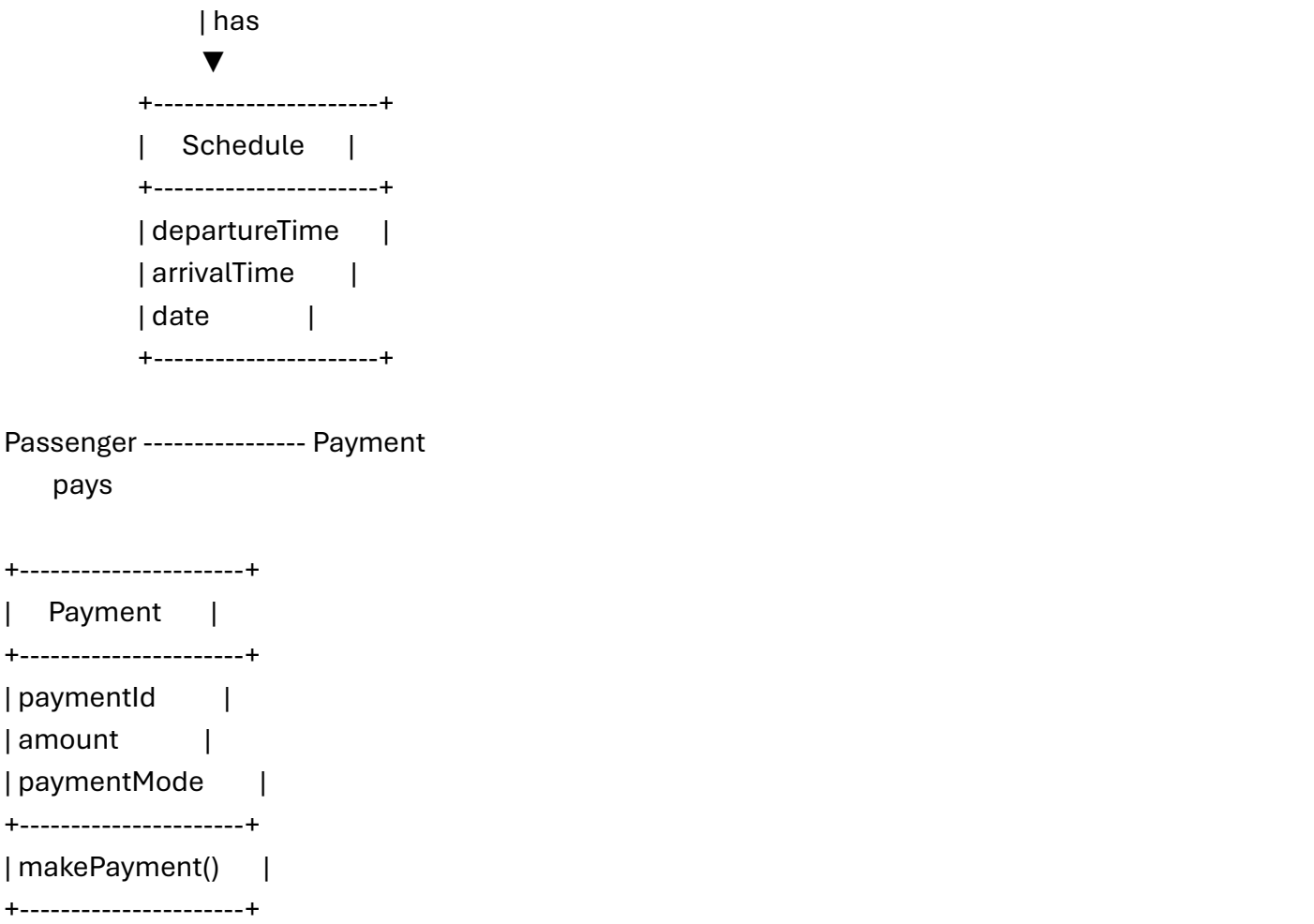
(SPPU BE OOMD – Simple Answer)

Definition

A **Class Diagram** represents the **static structure** of the Railway Reservation System by showing classes, attributes, methods, and relationships.

UML Class Diagram





Explanation of Classes

1. Passenger

- Stores passenger details.
- Books or cancels tickets.

2. Ticket

- Stores ticket number, seat number, fare, and booking status.

3. Train

- Stores train details such as train number, source, and destination.

4. Schedule

- Stores departure time, arrival time, and travel date.

5. Payment

- Handles ticket payment and stores payment information.

Relationships

- **Passenger → Ticket** : Passenger books a ticket.
- **Ticket → Train** : Ticket belongs to a train.
- **Train → Schedule** : Train has a schedule.
- **Passenger → Payment** : Passenger makes payment.

Advantages

- Represents the static structure of the railway reservation system.
- Clearly identifies classes and their relationships.
- Simplifies software design and maintenance.

Q2 (a) Explain different methods of identifying Analysis Classes. [7 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

Analysis Classes are the classes identified during the **Object-Oriented Analysis (OOA)** phase. They represent the important objects, data, and responsibilities of the system. Identifying analysis classes is one of the first steps in object-oriented design.

There are several methods used to identify analysis classes.

---

1. Noun Phrase (Grammatical) Method

Definition

In this method, the **requirement specification is read carefully**, and all **nouns** are identified. These nouns are considered as possible classes.

Steps

- Read the problem statement.
- Identify all nouns.
- Remove duplicate or unnecessary nouns.
- Select meaningful nouns as classes.

Example

Requirement:

"A student registers for a course."

Possible classes:

- Student
- Course

---

2. Use Case-Based Method

Definition

In this method, **use cases** are analyzed to identify the classes involved in each use case.

Steps

- Identify actors and use cases.
- Find the objects participating in each use case.
- Convert these objects into analysis classes.

Example

ATM Use Case:

- Customer
- ATM
- BankAccount
- Transaction

---

3. CRC (Class–Responsibility–Collaboration) Method

Definition

CRC stands for **Class, Responsibility, and Collaboration**. This method identifies classes by assigning responsibilities and determining how classes collaborate with each other.

CRC Card Format

Class	Responsibilities	Collaborators
Student	Register course	Course
Course	Maintain course details	Student

Advantages

- Clearly defines class responsibilities.
- Improves communication among developers.

---

4. Common Class Pattern Method

Definition

This method identifies classes using common patterns found in most software systems.

Common patterns include:

- Person
- Organization
- Place
- Event
- Transaction
- Device

Example

Hospital System:

- Patient
- Doctor
- Hospital
- Appointment

---

5. Category-Based Method

Definition

In this method, classes are identified by grouping them into categories such as:

- Physical objects
- Events
- Roles
- Places
- Organizations
- Transactions

Example

Library System:

- Book (Physical Object)
- Student (Role)
- Library (Organization)
- Issue (Transaction)

Summary Table

Method	Description	Example
Noun Phrase Method	Identify nouns from requirements	Student, Course
Use Case Method	Identify classes from use cases	Customer, ATM

Method	Description	Example
CRC Method	Identify classes using responsibilities	Student–Course
Common Class Pattern	Use common object patterns	Patient, Doctor
Category-Based Method	Group classes by categories	Book, Library

---

Advantages of Identifying Analysis Classes

- Helps understand system requirements.
- Forms the basis for class diagrams.
- Improves software design.
- Reduces development errors.
- Makes the system modular and reusable.

Conclusion

Different methods such as **Noun Phrase Method, Use Case Method, CRC Method, Common Class Pattern Method, and Category-Based Method** help identify analysis classes accurately. Choosing the correct classes leads to better object-oriented design and software quality.

Q2 (b) Differentiate between Class Diagram and Object Diagram in UML. [5 Marks]

A Class Diagram represents the abstract blueprint and static structure of a system, while an Object Diagram represents a concrete snapshot of that system at a specific moment in time.

Think of a class diagram as an architectural blueprint of a house and an object diagram as a photograph of a specific room inside that house once it is built. [1, 2]

Core Differences

The table below contrasts the primary differences between the two UML structural diagrams: [1, 2]

Feature	Class Diagram	Object Diagram
Concept	Abstract system model.	Real-world runtime instance.
Time Dependency	Independent of time.	Snapshot of a specific moment.
Main Component	Classes with attributes and methods.	Object instances with assigned values.
Notation Name	Named plain (e.g., Car).	Named and underlined (e.g., <u>myCar:Car</u>).
Data Values	Shows data types (e.g., int).	Shows actual values (e.g., speed = 60).
Relationships	Associations with multiplicities.	Links without multiplicities.

Detailed Breakdown

1. Representation & Abstraction

- **Class Diagram:** Focuses on system architecture and design. It outlines what data types and methods will exist.
- **Object Diagram:** Focuses on runtime behavior and system state. It tests the logic of your abstract structure against a realistic scenario. [1, 2, 3, 4, 5, 6]

2. Visual Notation Elements

- **Class Element:** A rectangle split into three sections: Class Name, Attributes, and Operations/Methods. [1, 2]
- **Object Element:** A rectangle split into two sections: Object Name (underlined with its associated class) and the current Slots/Attribute values. [1, 2, 3, 4, 5]
- **Connection Lines:** Class diagrams use structural relationships like inheritance, aggregation, and composition. Object diagrams use simple **Links** to show active connections between instances. [1, 2, 3, 4]

3. Purpose & Use Cases

- **Class Diagram:** Used to generate code, establish relationships, and design databases.
- **Object Diagram:** Used for debugging, system testing, and understanding complex data structures. [1, 2, 3, 4, 5]

If you want to dive deeper, let me know if you would like an **example scenario mapped out** using both diagram styles or if you need help with **UML notation symbols** like access modifiers.